



Università degli Studi dell'Insubria
Dipartimento di Scienze Teoriche e Applicate

Laboratorio Interdisciplinare A

Introduzione a Git e GitHub

Loris Bozzato

Dipartimento di Scienze Teoriche e Applicate

Perché Git?

Git è un **sistema di controllo di versione:**

- Permette di tenere traccia dei cambiamenti al codice
- Facilita la collaborazione di più persone sullo stesso progetto

Git è una **soluzione per lo sviluppo distribuito di software:**

- Più persone possono lavorare allo stesso codice in modo indipendente
- Possono accedere allo stesso repository di codice da più posizioni

(Queste slides riassumono la guida *Learn Git in 10 easy steps*, disponibile su Elearning, <https://medium.com/@letthedataconfess/learn-git-in-just-10-easy-steps-2d852d3172ea>)

Cos'è Git?



Git è un Distributed Version Control system **Open source e Free**:

<https://git-scm.com/>

- Permette a **più sviluppatori** di accedere ai codici sorgente
- Le **modifiche** al codice sono visibili a tutti gli sviluppatori
- **Version control**: più versioni del progetto, possibilità di tornare a una versione precedente
- **Repository**: cartella condivisa che contiene i file di un progetto

Cos'è GitHub?



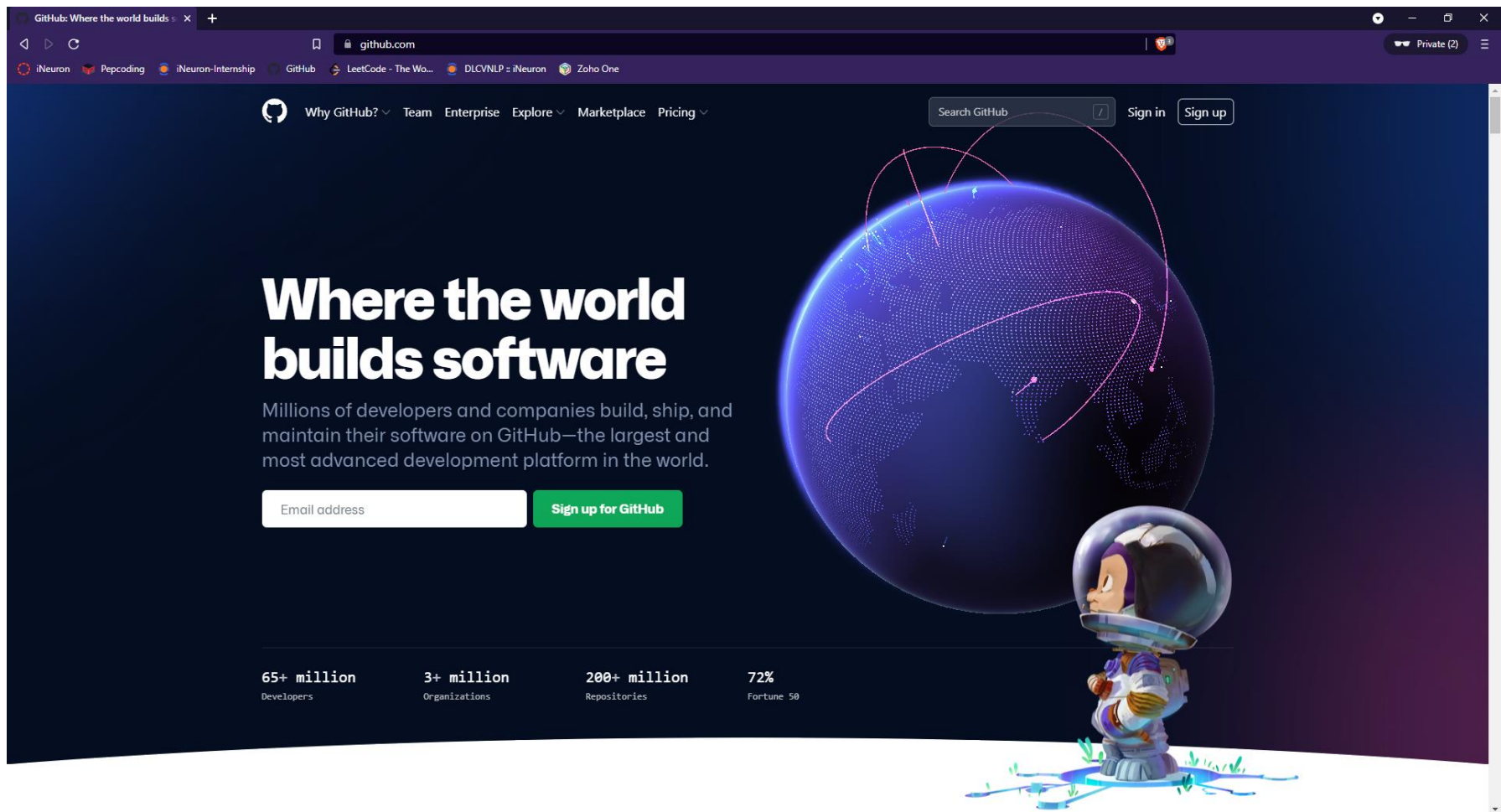
GitHub fornisce una implementazione di Git:

<https://github.com/>

GitHub è un **servizio di hosting di repository Git** cloud-based

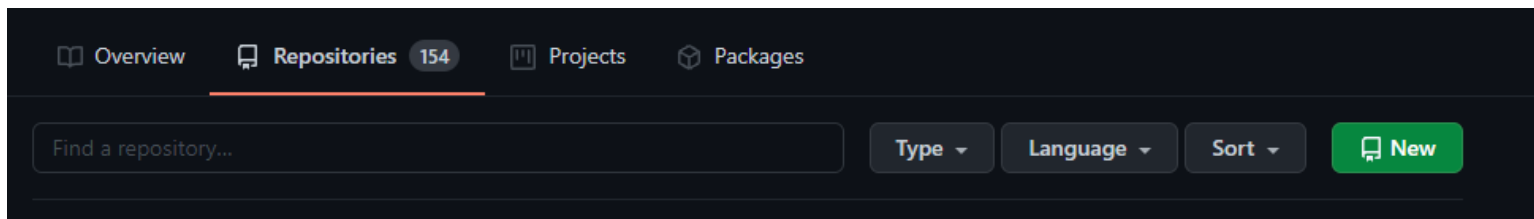
Creare un account GitHub

Per poter usare GitHub, registrarsi su <https://github.com/>



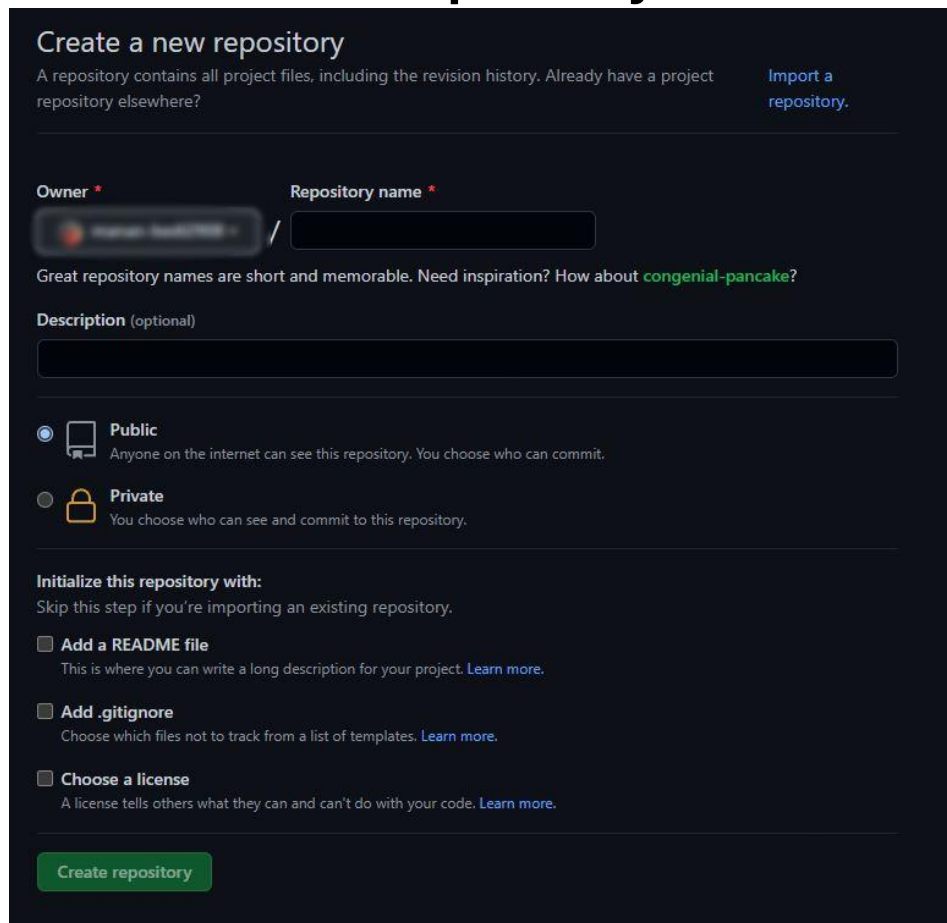
Creare un repository su GitHub

1. **Login** sul vostro account GitHub
2. Selezionate **New Repository** da Repositories



Creare un repository su GitHub

3. Scegliete il **nome** del repository da creare
4. Selezionate se rendere il repository **pubblico o privato**
5. Confermare con **Create Repository**



Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner * **Repository name ***

Great repository names are short and memorable. Need inspiration? How about [congenial-pancake?](#)

Description (optional)

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more.](#)

☐ **Add .gitignore**
Choose which files not to track from a list of templates. [Learn more.](#)

☐ **Choose a license**
A license tells others what they can and can't do with your code. [Learn more.](#)

Create repository

Istallare e configurare Git

I repository di GitHub si possono gestire dal vostro PC usando un **client Git**:

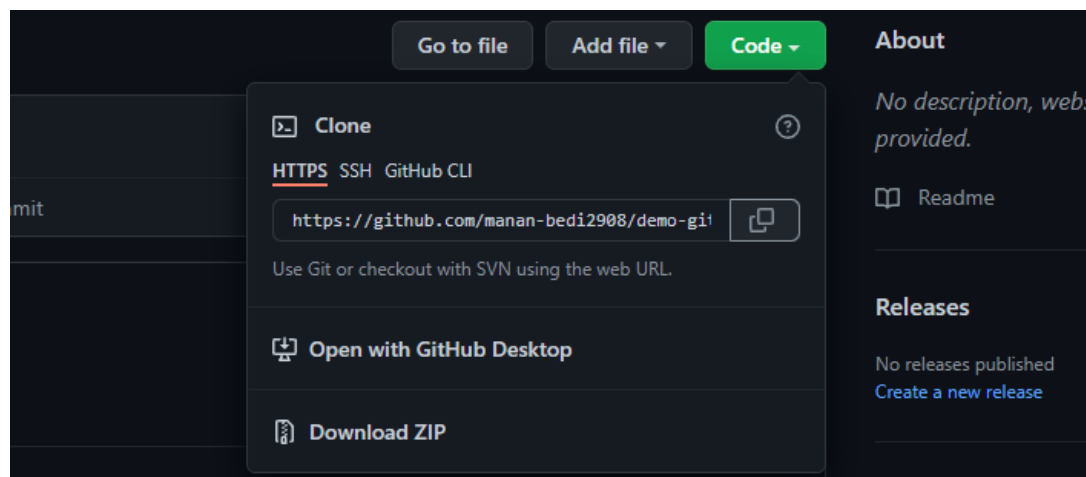
- **Installate** il client Git che preferite per il vostro SO:
<https://git-scm.com/downloads>
(Git da terminale o client GUI come GitHub desktop, Sourcetree...)
- **Configuratelo** per l'accesso a GitHub con le vostre credenziali:
Per esempio, da terminale:

```
$ git config --global user.name "YOUR_USERNAME"  
$ git config --global user.email "YOUR_EMAIL"
```


Clonare un repository GitHub

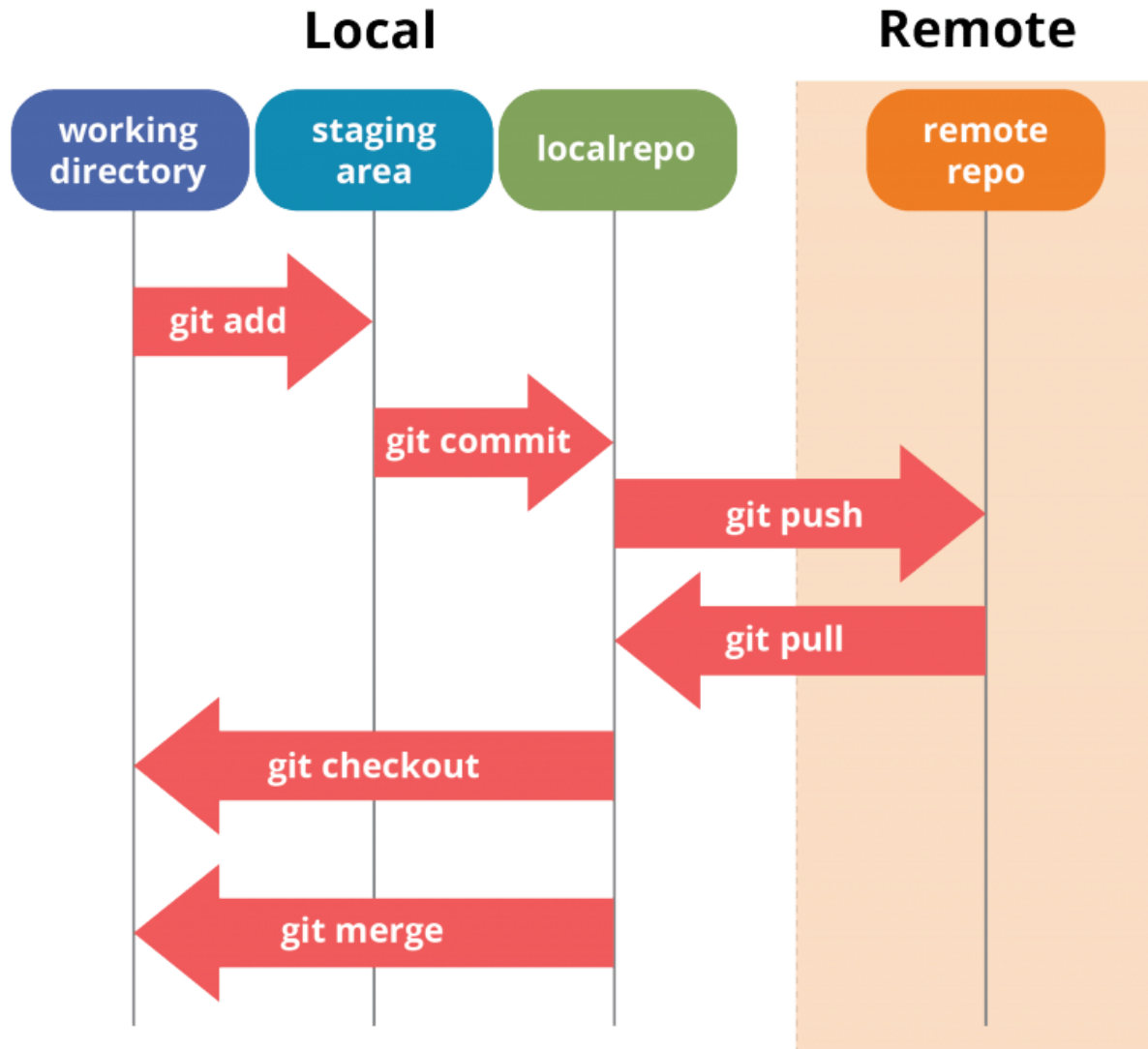
Per poter scaricare in locale un repository creato su GitHub, si può usare l'operazione **clone** di Git:

1. Andate alla pagina del repository GitHub da scaricare
2. Cliccate su **Code**



3. Per scaricarlo tramite Git, **scegliete il link HTTPS** che appare
4. Nel client Git, **eseguite una clone sul link fornito**
Per esempio, da terminale: `$ git clone <link>`
5. Il repository verrà "clonato" nella vostra working directory

Workflow di Git



Aggiungere modifiche: commit e push

Per aggiungere nuove modifiche al vostro repository GitHub:

- Se dovete **aggiungere un nuovo file al repository**, operazione di **Git add** (segnala a Git che deve tenere traccia del nuovo file)

```
$ git add "hello.java"
```

- Per **segnalare nuove modifiche** ai file (cioè creare una nuova "versione"), eseguite una operazione di **Git commit**

```
$ git commit -m "messaggio"
```



- Per aggiungere le modifiche al vostro **repository remoto** (GitHub), dopo la commit, eseguire un'operazione di **Git push**

```
$ git push origin main
```

Scaricare ultima versione: checkout

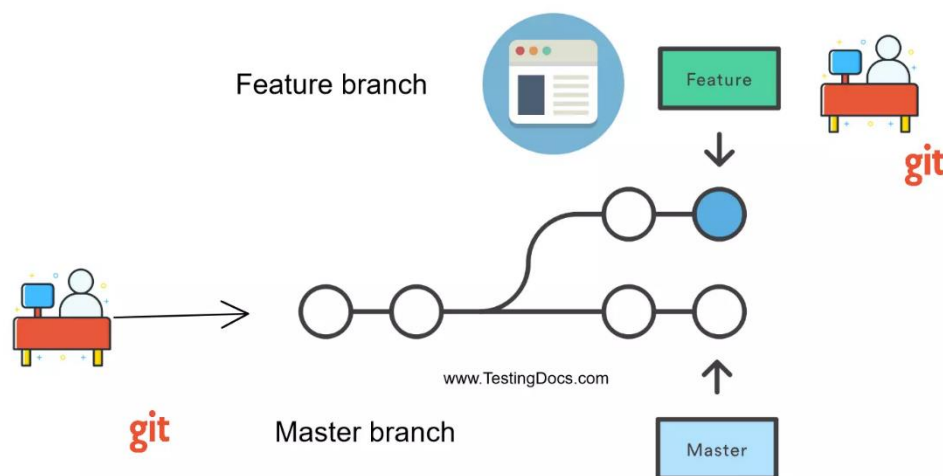
Viceversa, scaricare nuove versioni del codice da GitHub:

- Se dovete **scaricare l'ultima versione** del codice da GitHub, eseguite un'operazione di **Git checkout** (pull + checkout)

```
$ git checkout .
```

Diramazioni nelle versioni: branching

In Git, si possono avere **più "diramazioni" sulle versioni (branch)** per lavorare su diversi moduli del progetto



- Per **creare** un nuovo branch, usare l'operazione **Git branch**
`$ git branch nomebranch`
- Per **spostarsi** sul nuovo branch, operazione **Git checkout**
`$ git checkout nomebranch`

Adesso le modifiche che fate nella vostra copia locale andranno sulla nuova "diramazione" delle versioni del progetto

Ricongiungere i branch: merging

Si possono poi "**ricongiungere**" due **diramazioni**, per esempio per unire al branch principale (main) le modifiche di un branch secondario

- **Spostarsi** sul branch "di arrivo", con operazione **Git checkout**
`$ git checkout main`
- **Unire** il branch, con operazione **Git merge**
`$ git merge nomebranch`

In questo modo, la diramazione sparisce e le modifiche dei due branch vengono "fuse" nel branch di arrivo

E molto altro...

Questa è solo una introduzione alle operazioni più comuni, ma Git offre molte funzioni per la gestione dei repository, ad esempio:

- Scegliere come gestire i conflitti sui files
- Tornare a versioni precedenti
- Aggiungere tags a versioni specifiche
- Gestire più branch...

Per altre informazioni: pagine di **documentazione di Git e GitHub**:

<https://git-scm.com/doc>
<https://docs.github.com/en>